

Use Case: Sistem Kasir Laundry
Nama Mahasiswa: Muhamad Azmi Ma'mun
NIM / Username GitHub: nikolkanaeru

BAGIAN A — DOKUMEN ANALISIS

1. Problem Statement

Usaha laundry membutuhkan sistem terintegrasi yang dapat mengelola alur proses pengerjaan cucian (Cuci $\rightarrow$ Pengeringan $\rightarrow$ Setrika) secara berurutan dan terdisiplin, sekaligus menghitung total biaya transaksi berdasarkan tarif kuintal/berat, batas minimum berat, jenis layanan tambahan (express), serta skenario diskon bagi pelanggan.

2. Actor

- **Petugas Laundry / Kasir:** Menginput data transaksi pelanggan, memeriksa status diskon, serta mencatat progress alur pengerjaan setiap tahap laundry.
- **Pelanggan:** Memproses pembayaran transaksi berdasarkan perhitungan berat dan promo diskon yang berlaku.

3. Input & Output

- **Input:**
 - Data Pelanggan (ID, Nama, Status Member).
 - Data Transaksi (Berat cucian dalam kg, Jenis Layanan: Reguler / Express).
 - Data Alur Proses (Kode tahap, durasi pengerjaan dalam menit, status penyelesaian true/false).
- **Output:**
 - Rincian perhitungan biaya laundry (Berat efektif, Biaya dasar, Biaya express, Persentase diskon, Nominal potongan, Total bayar).
 - Pesan status penyelesaian alur proses laundry per tahap beserta catatan keterlambatan.

4. Functional Requirement

- **FR-01:** Sistem dapat menghitung biaya transaksi laundry berdasarkan berat, jenis layanan, dan keanggotaan (member).
- **FR-02:** Sistem dapat memvalidasi dan memproses alur pengerjaan laundry tahap demi tahap (Cuci $\rightarrow$ Pengeringan $\rightarrow$ Setrika).
- **FR-03:** Sistem menampilkan konfirmasi hasil transaksi dan laporan alur proses secara mendetail.

5. Business Rules (Aturan Bisnis)

- **BR-01 (Tarif Dasar):** Tarif standar laundry adalah **Rp7.000 / kg**.
- **BR-02 (Batas Berat Minimum):** Berat cucian di bawah 2 kg dihitung secara otomatis sebagai 2 kg.
- **BR-03 (Layanan Express):** Layanan express dikenakan biaya tambahan sebesar **+50%** dari biaya dasar.
- **BR-04 (Validasi Input Berat):** Berat cucian yang diinput harus lebih besar dari 0 kg ($\text{berat} > 0$).
- **BR-05 (Aturan Diskon Transaksi):**
 - Minimal total belanja Rp100.000 mendapat diskon **10%**.
 - Pelanggan dengan status **Member** mendapat tambahan diskon **5%**.
 - Batas maksimal persentase diskon adalah **15%**.
 - Batas maksimal nominal potongan harga adalah **Rp25.000**.
- **BR-06 (Urutan Tahapan Pengerjaan):** Alur pengerjaan laundry harus berurutan (Tahap 1: Cuci, Tahap 2: Pengeringan, Tahap 3: Setrika & Lipat). Jika melompati urutan, proses ditolak.
- **BR-07 (Batas Waktu Operasional):** Jika durasi pengerjaan melebihi batas (Cuci > 45 menit, Pengeringan > 30 menit, Setrika > 20 menit), sistem mencatat keterlambatan.

6. Decomposition

Sistem dipecah menjadi beberapa fungsi spesifik (satu fungsi = satu tugas):

- **Perhitungan Transaksi & Diskon:**
 1. `hitungBeratDihitung(double beratAsli)` $\rightarrow$ Menerapkan aturan minimum berat 2 kg (BR-02).
 2. `hitungBiayaDasar(double beratDihitung)` $\rightarrow$ Menghitung biaya dasar Rp7.000/kg (BR-01).
 3. `hitungBiayaLayanan(double biayaDasar, JenisLayanan layanan)` $\rightarrow$ Menghitung surcharge express 50% (BR-03).
 4. `hitungPersenDiskon(double totalBelanja, bool isMember)` $\rightarrow$ Menghitung persentase diskon 10% + 5% max 15% (BR-05).
 5. `hitungTotalBayar(double totalBelanja, bool isMember)` $\rightarrow$ Menerapkan batas maksimal potongan Rp25.000 (BR-05).
- **Alur Pengerjaan (Workflow):**
 1. `tahapCuci(int tahap, int waktu, bool selesai)` $\rightarrow$ Validasi & pengerjaan Tahap 1.
 2. `tahapKering(int tahap, int waktu, bool selesai)` $\rightarrow$ Validasi & pengerjaan Tahap 2.
 3. `tahapSetrika(int tahap, int waktu, bool selesai)` $\rightarrow$ Validasi & pengerjaan Tahap 3.

7. Pattern Recognition

- **Validasi Urutan & Guard Clause:** Menggunakan pola *early return* (`if (kondisiGagal) return false;`) pada setiap tahap alur proses pengerjaan untuk menghentikan eksekusi jika urutan salah atau belum selesai.
- **Diskon & Cap Limit Pattern:** Pola akumulasi syarat bertahap yang diakhiri pembatasan batas atas (*capping*) baik pada persentase (`min/max`) maupun nominal rupiah.

8. Abstraction

- **Enum:**
 - `JenisLayanan: { regular, express }`
 - `StatusTahap: { cuci, pengeringan, setrika }`
- **Class TransaksiLaundry:**
 - `id: String`
 - `namaPelanggan: String`
 - `beratAsli: double`
 - `beratDihitung: double`
 - `layanan: JenisLayanan`
 - `isMember: bool`
 - `totalBayar: double`

9. Algorithm

PROCEDURE tahapCuci(tahap, waktu, selesai)

IF tahap != 1 THEN

 DISPLAY "tidak sesuai urutan proses"

 RETURN false

END IF

IF selesai == false THEN

 DISPLAY "Tahap 1 (Cuci) belum selesai"

 RETURN false

END IF

IF waktu > 45 THEN

 DISPLAY "Proses pencucian memakan waktu terlalu lama (terlambat)"

END IF

DISPLAY "Tahap 1 (Cuci) sudah selesai"

RETURN true

END PROCEDURE

PROCEDURE tahapKering(tahap, waktu, selesai)

IF tahap != 2 THEN

 DISPLAY "tidak sesuai urutan proses"

 RETURN false

END IF

IF selesai == false THEN

 DISPLAY "Tahap 2 (Pengeringan) belum selesai"

 RETURN false

END IF

IF waktu > 30 THEN

 DISPLAY "Proses pengeringan memakan waktu terlalu lama (terlambat)"

END IF

DISPLAY "Tahap 2 (Pengeringan) sudah selesai"

RETURN true

END PROCEDURE

```
PROCEDURE tahapSetrika(tahap, waktu, selesai)
```

```
  IF tahap != 3 THEN
```

```
    DISPLAY "tidak sesuai urutan proses"
```

```
    RETURN false
```

```
  END IF
```

```
  IF selesai == false THEN
```

```
    DISPLAY "Tahap 3 (Setrika & Lipat) belum selesai"
```

```
    RETURN false
```

```
  END IF
```

```
  IF waktu > 20 THEN
```

```
    DISPLAY "Proses penyetrikaan memakan waktu terlalu lama (terlambat)"
```

```
  END IF
```

```
  DISPLAY "Tahap 3 (Setrika & Lipat) sudah selesai. Laundry siap diambil!"
```

```
  RETURN true
```

```
END PROCEDURE
```

```
PROCEDURE main()
```

```
  DISPLAY "=== PROSES ALUR LAUNDRY ==="
```

```
  hasilTahap1 = tahapCuci(1, 30, true)
```

```
  IF hasilTahap1 == true THEN
```

```
    hasilTahap2 = tahapKering(2, 25, true)
```

```
  IF hasilTahap2 == true THEN
```

```
    tahapSetrika(3, 15, true)
```

```
  ELSE
```

```
    DISPLAY "Tahap 2 (Pengeringan) belum selesai"
```

```
  END IF
```

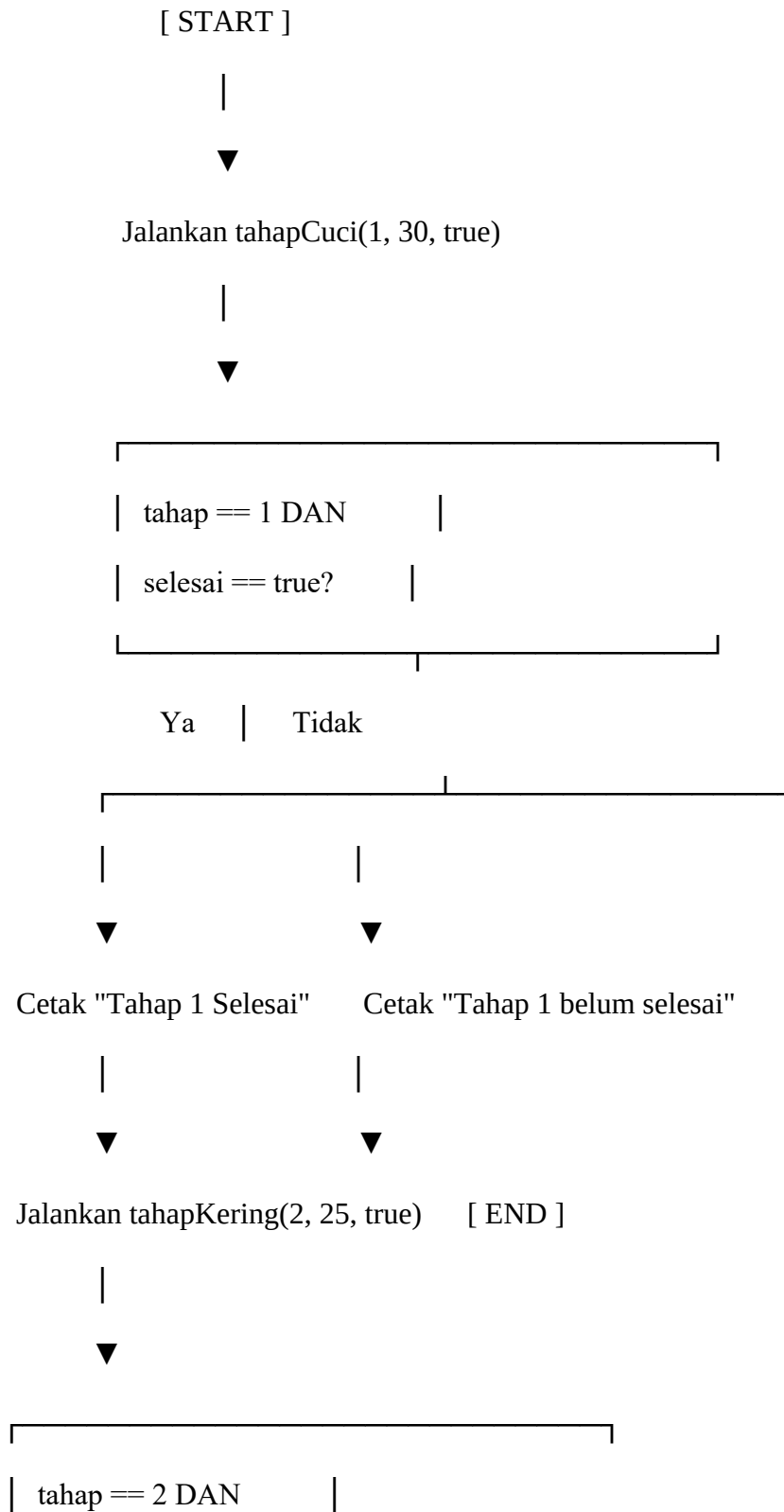
```
ELSE
```

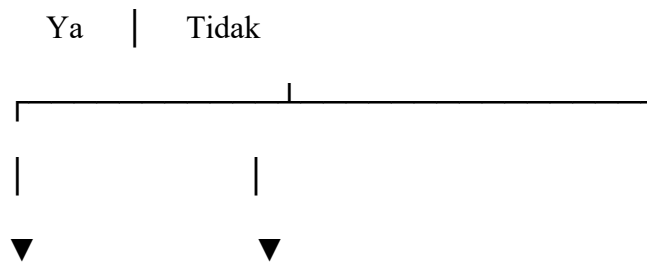
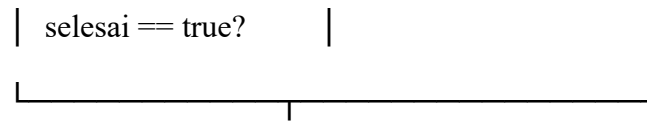
```
  DISPLAY "Tahap 1 (Cuci) belum selesai"
```

```
END IF
```

```
END PROCEDURE
```

10. Flowchart

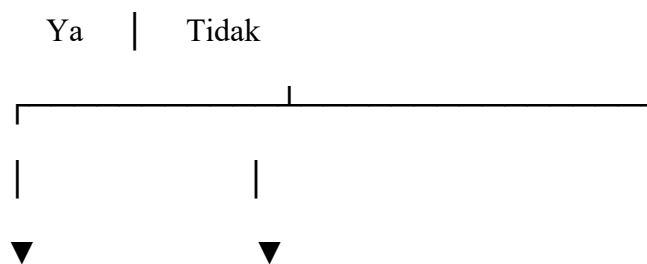
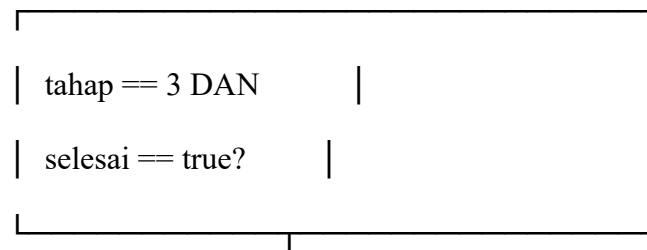




Cetak "Tahap 2 Selesai" Cetak "Tahap 2 belum selesai"

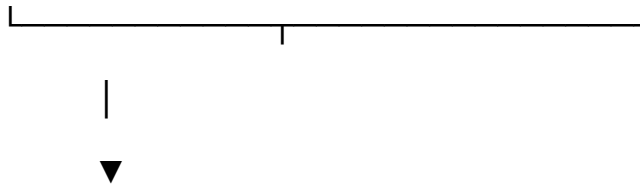


Jalankan tahapSetrika(3, 15, true) [END]



Cetak "Tahap 3 Selesai & Cetak "Tahap 3 belum selesai"





[END]

BAGIAN C — TABEL TRACEABILITY

Business Rule	Deskripsi Aturan Bisnis	Function yang Mengimplementasikan	Skenario Uji Pada Kode (main)
BR-01	Validasi urutan tahap (Tahap 1, 2, 3 harus berurutan)	<code>tahapCuci()</code> , <code>tahapKering()</code> , <code>tahapSetrika()</code>	Parameter tahap diisi 1, 2, dan 3 secara berurutan
BR-02	Validasi status selesai (<code>selesai == true</code>)	<code>tahapCuci()</code> , <code>tahapKering()</code> , <code>tahapSetrika()</code>	Parameter selesai diperiksa dengan percabangan <code>if-else</code>
BR-03	Deteksi keterlambatan waktu pencucian (<code>waktu > 45</code>)	<code>tahapCuci()</code>	Pengujian batas parameter waktu
BR-04	Deteksi keterlambatan waktu pengeringan (<code>waktu > 30</code>)	<code>tahapKering()</code>	Pengujian batas parameter waktu
BR-05	Deteksi keterlambatan waktu penyetrikaan (<code>waktu > 20</code>)	<code>tahapSetrika()</code>	Pengujian batas parameter waktu
BR-06	Hentikan alur (<i>Guard Clause</i>) jika tahap sebelumnya belum selesai	<code>main()</code>	Percabangan bersarang (<i>nested if</i>) <code>hasilTahap1</code> dan <code>hasilTahap2</code>

